

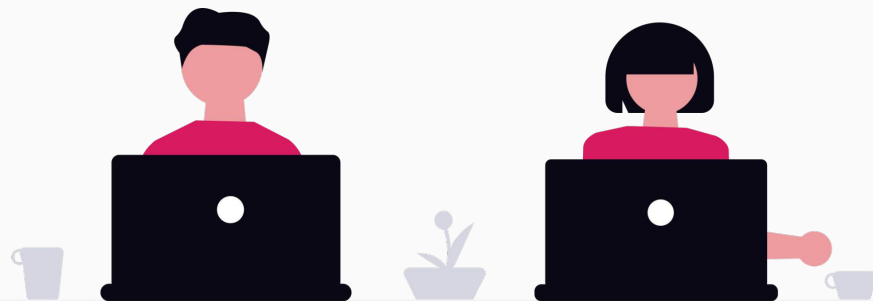


Hackerschool: Intro to Git

Tuesday, 07 October 2025, 1830h — 2030h

by Yik Jin (yikjin@nushackers.org)

Get these slides at hckr.cc/hs-git-intro



Prerequisites



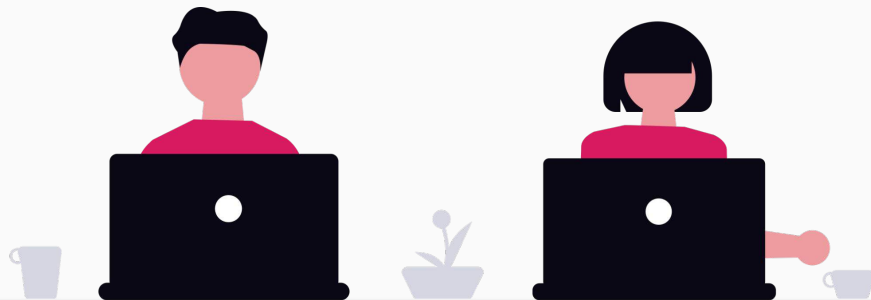
1. **Install Git** (from git-scm.com/downloads) on your operating system of choice
2. **Create an account** at github.com

We assume you have basic proficiency in terminal



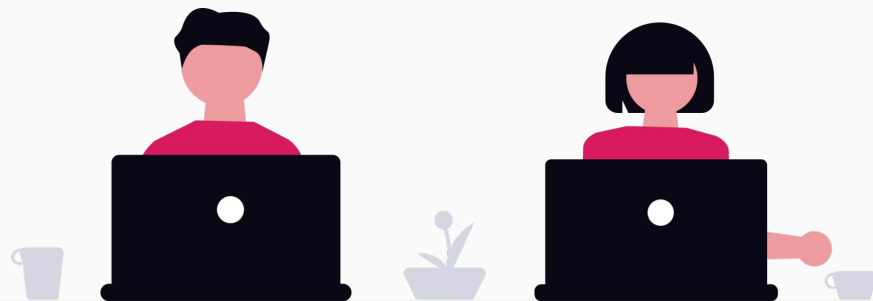
This will be a hands-on session!

There will be time for you to try out Git (and GitHub) later :)



Why learn Git?

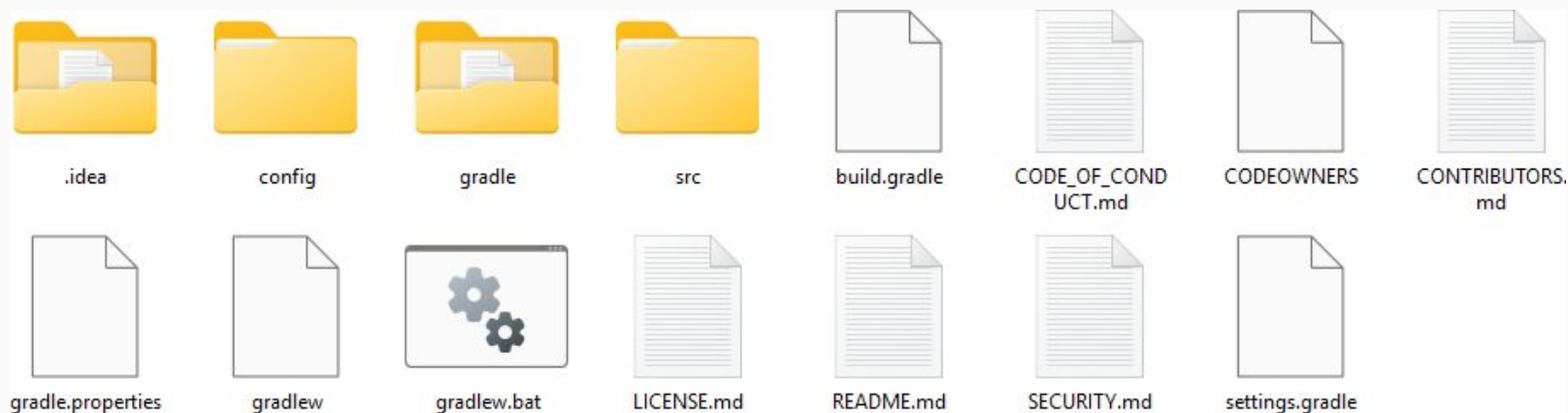
Get these slides at hckr.cc/hs-git-intro



Working on a project...



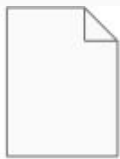
How will you **collaborate** with others?



Backing up project?



main - Copy.c



main.c



main_backup.c



main_working.c



src



src - Copy

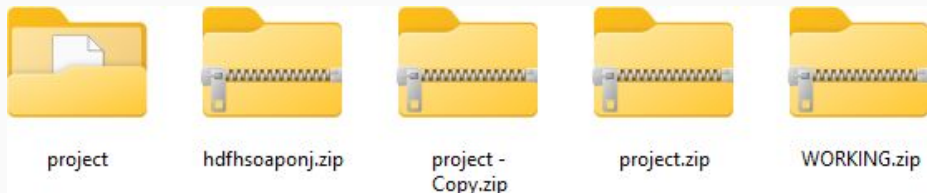


src - Copy -
Copy



src - Copy -
Copy - Copy

Sharing ZIPs with others?



What if you need to...



- **Work on different parts** of a project with others at the same time?
- **Revert parts of a file** to a specific point in time?
- Understand how your code **changes over time**?

If only there is a simpler and better way...



What if you need to...



- **Work on different parts** of a project with others at the same time?
- **Revert parts of a file** to a specific point in time?
- Understand how your code **changes over time**?

If only there is a simpler and better way...

Yes, there is!



Version control system (VCS)



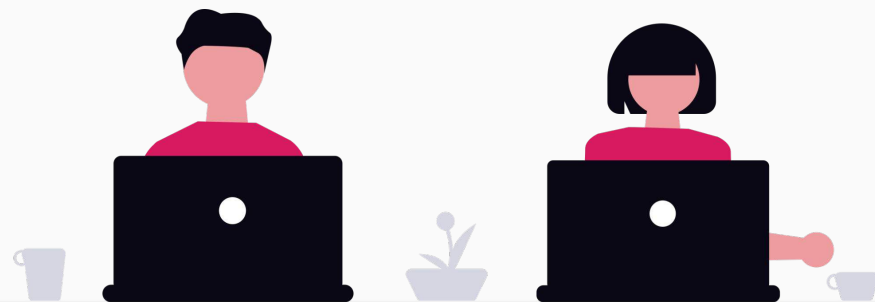
VCS is “software that **manages project changes**, allowing developers to store versions, compare updates, and revert when necessary”

Distributed VCS (DVCS) allows people to **work from multiple computers**, and **collaborate with others remotely**

- **Git** is an example of a DVCS

Intro to Git

Get these slides at hckr.cc/hs-git-intro



What is Git?



Git is a...

- **Command-line based** software tool that runs on your **local computer**
- **Tracker for changes** over time in a project
(not limited to just code)
- Manually invoked tool *(not a daemon/background process)*



What is Git?



Git is **not** an automated tool!

It **cannot** / **is not** meant to...

- Automatically save code changes in a project (*manual saving required*)
- Automatically give others access to your project (*you need a web service like GitHub for that*)
- Track files outside of your project (*only tracks files in Git repositories*)

Initialising a project



Git must be **initialised in a project** before using it

- Can be done either in a new project, or an existing one

A folder initialised with Git is known as a "**repository**" ("repo" in short)

Activity 1: Initialise a new repository



Initialise a new repository with Git

1. Open the **terminal** (e.g. Git Bash on Windows)
2. **Create** a new project folder (e.g. `mkdir ~/Documents/intro-to-git`)
3. **Change directory** into it (e.g. `cd ~/Documents/intro-to-git`)
4. Run `git init` to **initialise the repository**

If **all goes well**, you should see a message like: `Initialized empty Git repository in C:/Users/User/Documents/intro-to-git/.git/`

How Git keeps track of history



Git tracks how code in a repository evolves using "save points" known as "**commits**"

Commits track changes to **selected files and folders** in a repository

Commits must be **manually created** by you

We visualise a git commit as a dot

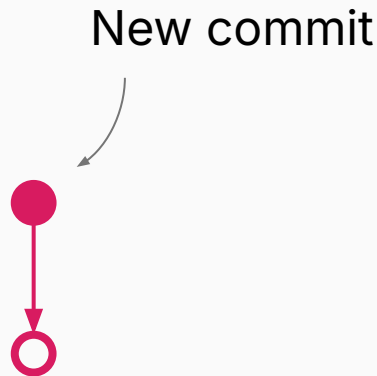


How Git keeps track of history



Commits **build on top** of other commits (*except for your first ever commit*)

- They **store files changed** from the previous commit



First time Git setup



Git needs to **know your details** before your first commit, so that it can **record them** in the commits that you make:

```
git config --global user.name "Full Name"
```

```
git config --global user.email user@example.com
```



Enter the email you used
to sign up for GitHub!

First time Git setup



Set up your **default editor** to use for Git as well:

```
git config --global core.editor code
```

Set up your **default branch** in Git (*we will cover this later, type in as-is*):

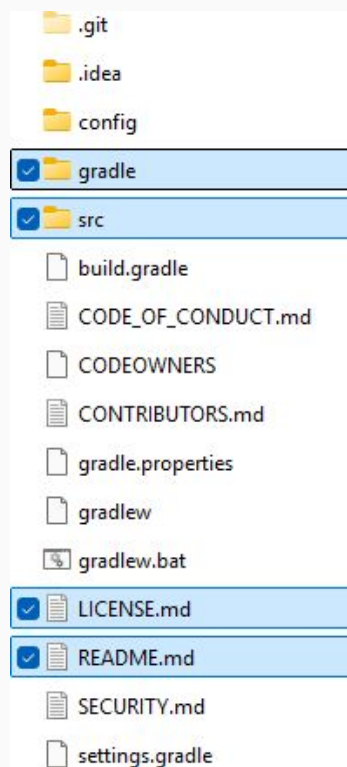
```
git config --global init.defaultBranch main
```

Committing in Git



Before you commit your files, you have to **select which files to save**

- The selected files becomes **"staged"** in the "staging area"
- Stage files with `git add {file(s)}`
- **See the status** of the files in the repository with `git status`



Committing in Git



To **commit** your staged files, run `git commit`

- The default text editor will open up to enter a **commit message**

Enter **a line of commit message**, and optionally paragraph(s) of descriptive text **with an empty line in between**:

```
chore(setup): add initial commit
```

```
Add initial commit files.
```

Viewing commits



To view your **list of commits**, run `git log`

- Check that your commit **appears**

```
commit d382d37dd838060eebe7aed065d3479c05a0cdf4 (HEAD -> main)
Author: Yik Jin <59534057+yikjin@users.noreply.github.com>
Date:   Tue Oct 7 13:03:35 2025 +0800

    chore(setup): add initial commit

    Add initial commit files.
```



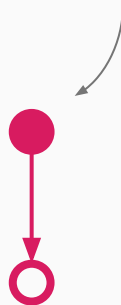
Activity 2: Creating commits



In your new repository:

1. **Create** a new file `README.md` (it can contain any text)
2. **Add the file** to the staging area, and **commit** it
3. **Update** the `README.md` file
4. **Add the file** to the staging area, and **commit** it again
5. Ensure you see the 2 commits created

Update `README.md`



Branching in Git

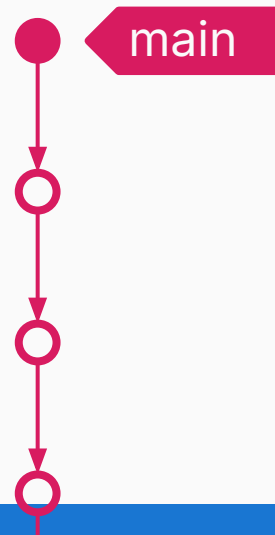


Notice that as you start to do more commits, a **path of commits** start to form

- These paths are known as "**branches**"

When you did your first commit, it was created in a branch called "**main**"

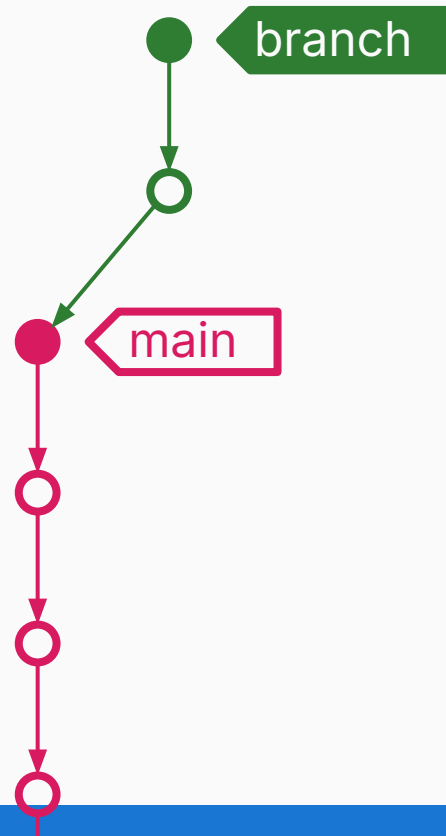
- The default branch used to be called "master" before 2020



**HACKER
SCHOOL**

- Considered **good practice** to use branches

- Use "main" branch as production / branch with good working code
- Use a branch for **each feature** you're working on



Creating a new branch

Create a branch named "**add-profile**" with
`git branch add-profile`

- The new branch points to the **same commit** as the "main" branch (*there are no commits in "add-profile" yet*)
- Your current **active branch** is still "main"!

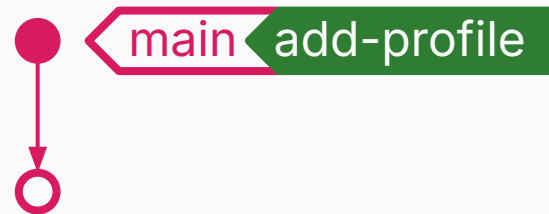


See your **current active branch** with
`git branch`

Creating a new branch

Switch to the newly-created branch with
`git switch add-profile`

- All **subsequent commits** that you make will be on the “add-profile” branch, not “main” anymore
- Alternatively, you can also **create and switch** to a new branch with
`git switch -c new-branch`

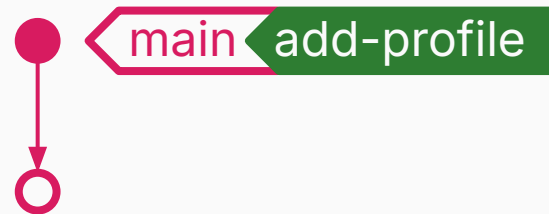


Adding commits to a branch



To **add commits** to your active branch, use the same commands as taught earlier:

1. Make some changes
2. Add changed files to **staging area** (use `git add -A` to add all changes)
3. **Commit** the staged files (with `git commit -m 'message'`)

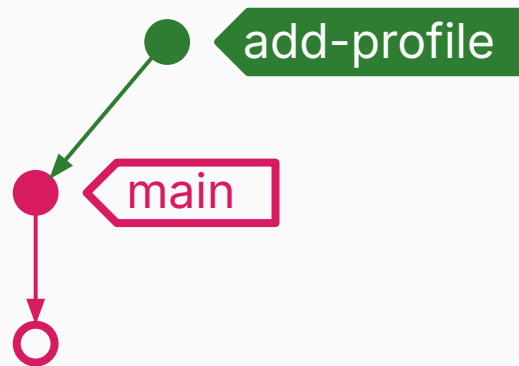


Adding commits to a branch



To **add commits** to your active branch, use the same commands as taught earlier:

1. Make some changes
2. Add changed files to **staging area** (use `git add -A` to add all changes)
3. **Commit** the staged files (with `git commit -m 'message'`)



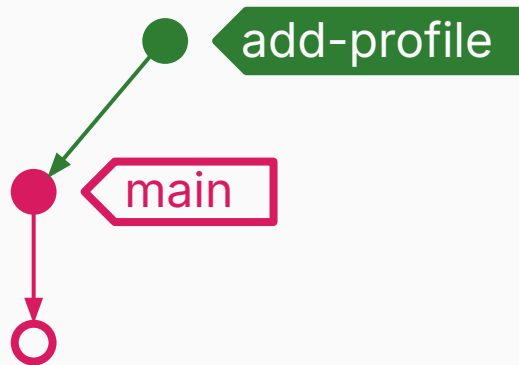
Viewing commits in branch



You can **view** the newly-created commit with
`git log --graph`

- See a **condensed view** with
`git log --graph --oneline`

```
* 713d44d (HEAD -> add-profile) docs(profile): add @yikjin profile
* 6f43d4f (main) docs(readme): add more information to `README.md`
* d382d37 chore(setup): add initial commit
```

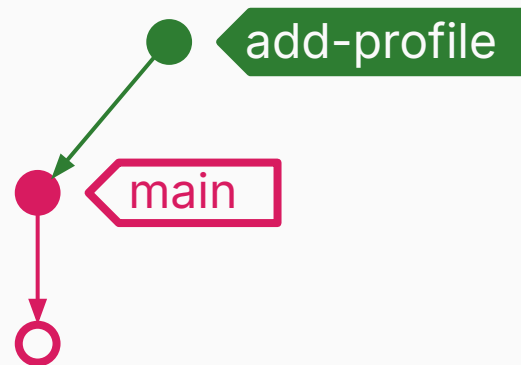


Activity 3: Committing in new branch



Now, it's your turn to **add commits** to the new branch!

1. **Create** a new branch "add-profile"
2. **Switch** to new branch
3. **Add** `{your_github_username}.md` file, and type your introduction!
4. **Stage** the file, and **commit** it
5. **Confirm** the commit is made correctly



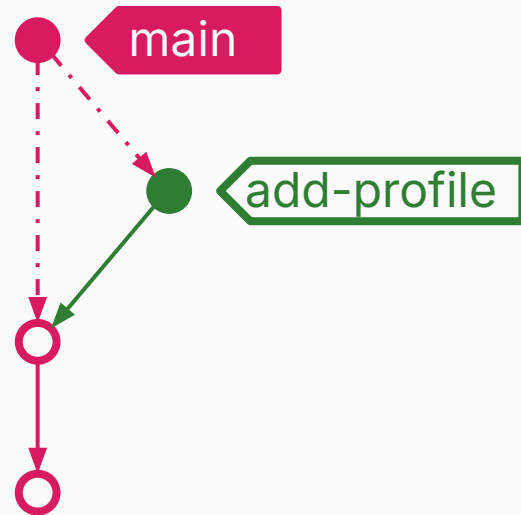
Merging branches



Great! You got the new feature of adding your profile ready for “production”. How do we make our commits in “add-profile” appear in “main”?

- Done by **merging branches** together

“add-profile” **source branch** is being merged into “main” **destination branch**

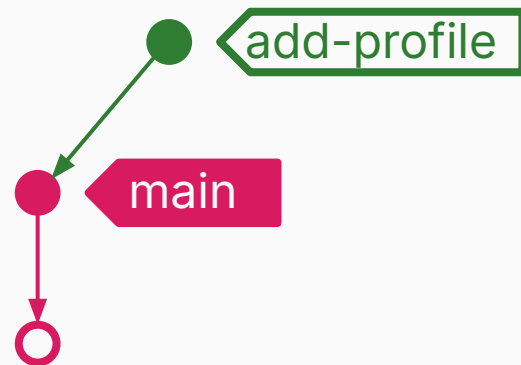


Fast-forward merge



By default, Git tries to do a **fast-forward merge**

- Used when **no new commits made** on the destination branch
- "Fast-forward" because Git just **pushes** the destination branch pointer **forward**

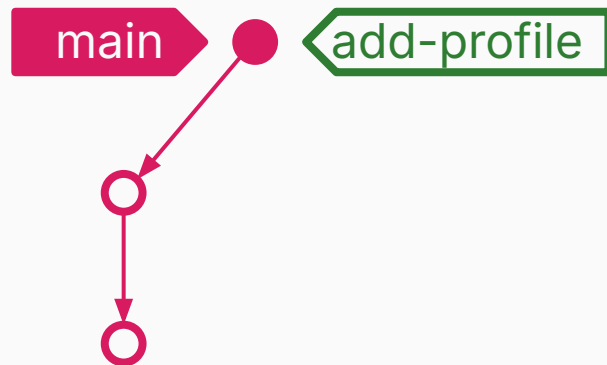


Fast-forward merge



By default, Git tries to do a **fast-forward merge**

- Used when **no new commits made** on the destination branch
- "Fast-forward" because Git just **pushes** the destination branch pointer **forward**



Both ways of merging will **combine changes** from both branches

-
- ```

graph TD
 A(()) --> B(())
 B --> C(())
 A --- A_label[add-profile]
 B --- B_label[main]
 style A fill:#006400,color:#fff
 style B fill:#DC143C,color:#fff
 style C fill:#DC143C,color:#fff

```

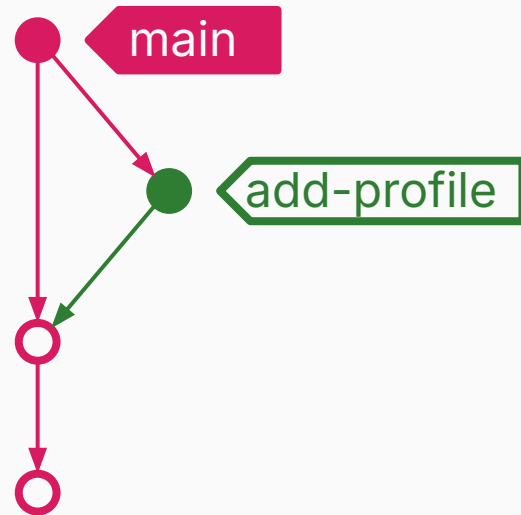
# No fast-forward merge



Git can also do a no fast-forward merge, which **creates a merge commit** with 2 parents

Both ways of merging will **combine changes** from both branches

- Fast-forward merge results in **linear history**, no fast-forward merge **keeps branching history**



# How to merge in Git



To merge "add-profile" into "main":

1. **Switch** to "main" branch
2. **Merge** "add-profile" branch with one of:

|                                  |                                                        |
|----------------------------------|--------------------------------------------------------|
| <code>git merge</code>           | <code>add-profile</code> (default: fast-forward merge) |
| <code>git merge --ff</code>      | <code>add-profile</code> (fast-forward merge)          |
| <code>git merge --ff-only</code> | <code>add-profile</code> (only use fast-forward merge) |
| <code>git merge --no-ff</code>   | <code>add-profile</code> (no fast-forward merge)       |

# Activity 4: Merging branches



Now it's your turn! **Merge** the "add-profile" branch to the "main" branch:

1. **Switch** to "main" branch (with `git switch main`)
2. **Merge** "add-profile" into "main" with any merge resolution
3. **Check** that your merge is correctly done!

```
* 713d44d (HEAD -> main, add-profile) docs(profile): add @yikjin profile
* 6f43d4f docs(readme): add more information to `README.md`
* d382d37 chore(setup): add initial commit
```

```
* 911adf1 (HEAD -> main) Merge branch 'add-profile'
| \
| * 713d44d (add-profile) docs(profile): add @yikjin profile
| /
* 6f43d4f docs(readme): add more information to `README.md`
* d382d37 chore(setup): add initial commit
```

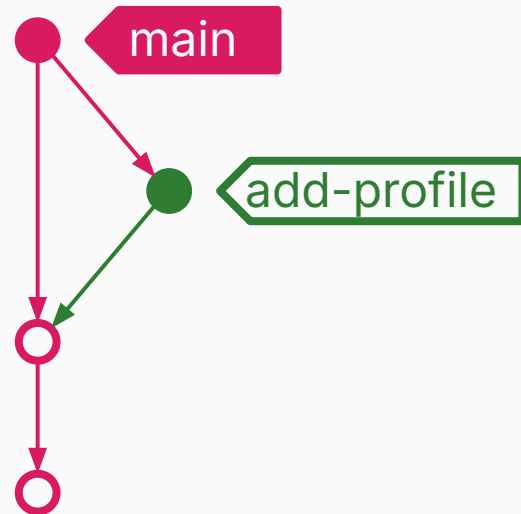
# Deleting branches



After merging the “add-profile” branch, if we no longer need it anymore, we can **delete the branch**

- No harm keeping it around, no harm deleting the branch

**Delete** the branch with  
`git branch -d add-profile`

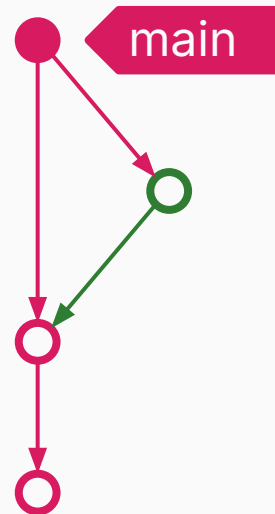


# Deleting branches

After merging the “add-profile” branch, if we no longer need it anymore, we can **delete the branch**

- No harm keeping it around, no harm deleting the branch

**Delete** the branch with  
`git branch -d add-profile`

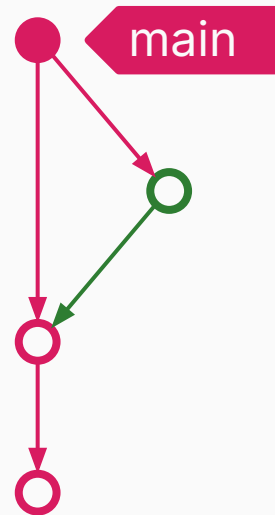


# Exploring old commits



What if you want to explore the codebase at a **previous commit**?

- You can **view the list** of commits with `git log`, and take note of the **commit hash** you want to go to
- Use `git switch -d {hash}` (first 4 or more characters will do)

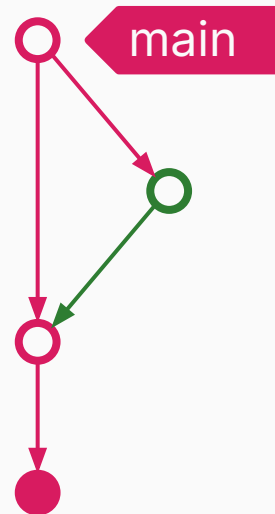


# Exploring old commits



What if you want to explore the codebase at a **previous commit**?

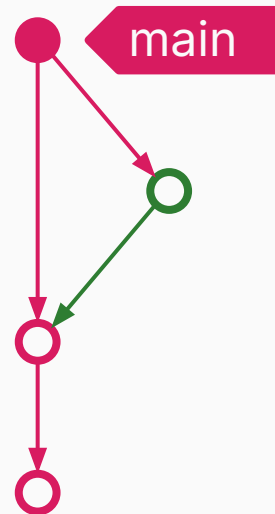
- You can **view the list** of commits with `git log`, and take note of the **commit hash** you want to go to
- Use `git switch -d {hash}` (first 4 or more characters will do)



# Going back to most recent commit



Once you are done exploring, **switch back** to a branch with `git switch {branch}`

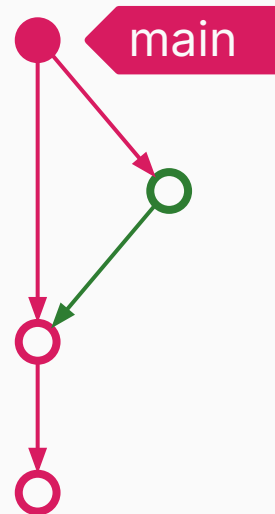


# Activity 5: Travelling through time



Try exploring your **previous commits**:

1. Explore your **old commits** with  
`git switch -d {hash}`
2. You can **see** you are at a set commit with  
`git status` and `git log`
3. Make sure it worked by **exploring** the old version of the files
4. **Go back** to the "main" branch once you are done with `git switch main`



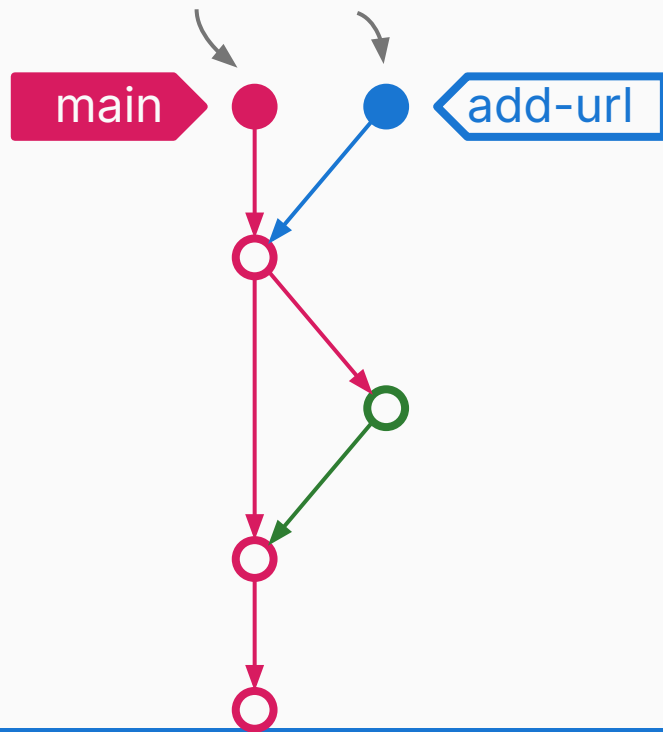
# A while later...



Now imagine that we created **more commits** in more branches... for example:

- We **change a line** in `{username}.md` in the "main" branch
- We **change the same line** in the **same file** in "add-url" branch

Both commits change the **same line** in the **same file**

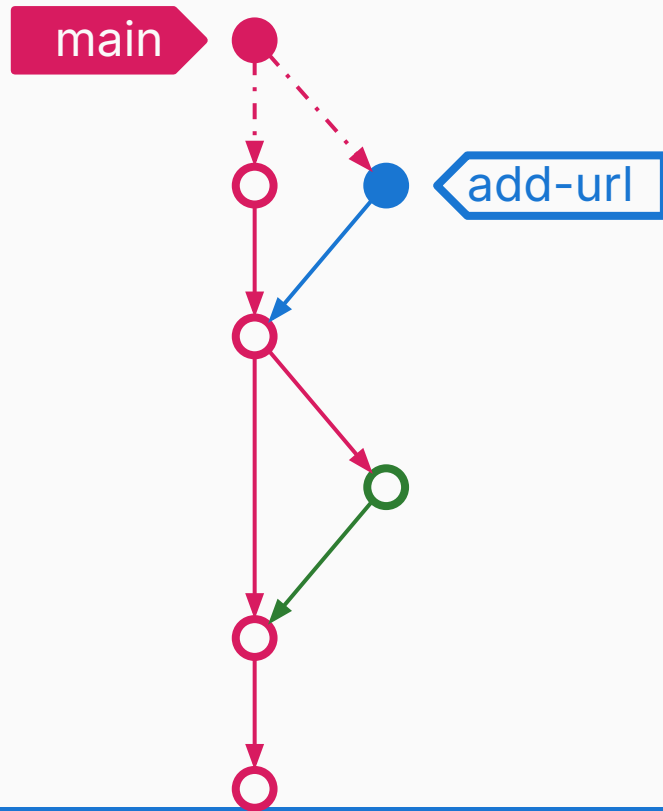


# What happens if we merge?



Now, what will happen if we merge both branches?

- There will be a **merge conflict!**



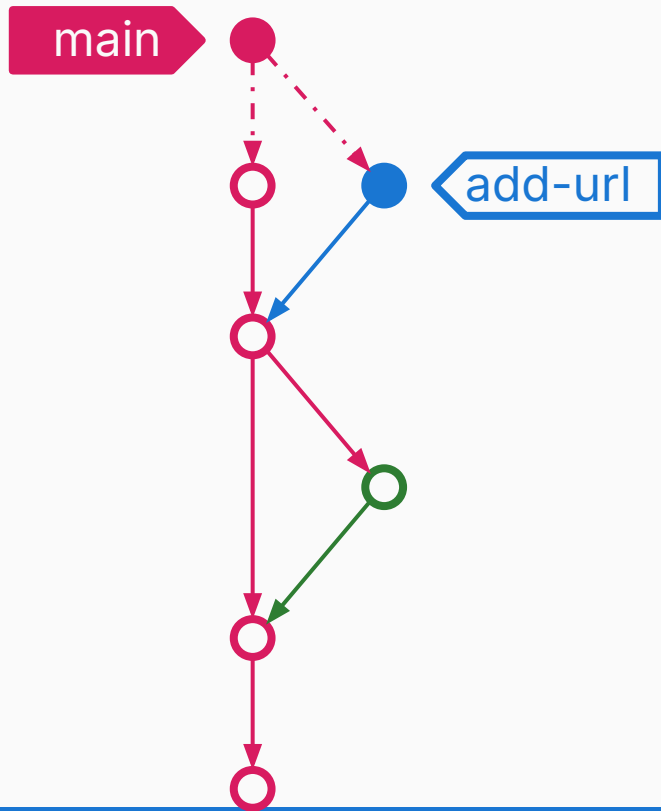
# Why do merge conflicts happen?



Git does not know how to **automatically resolve** the conflicting changes

Git will **highlight** the areas of conflict in the affected files

- You have to **manually resolve** them in your text editor to complete the merge

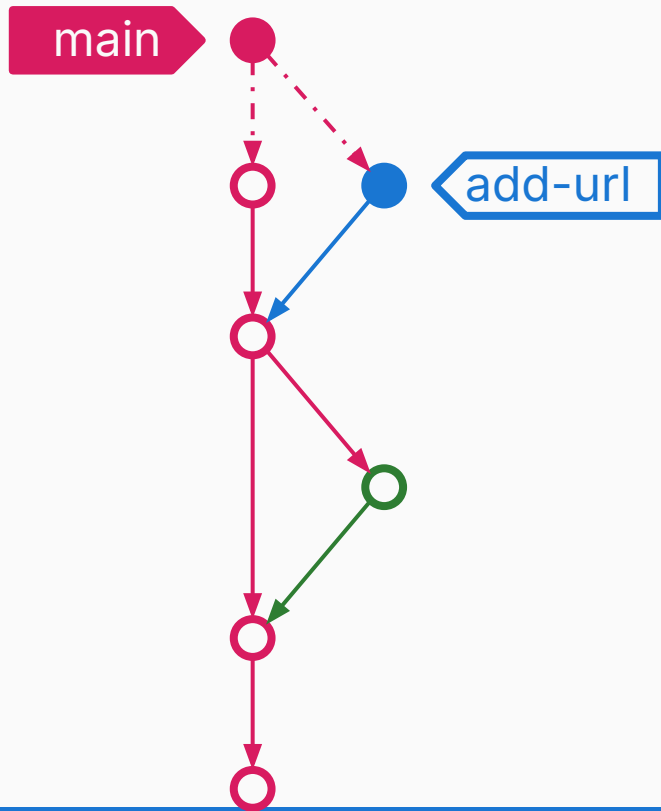


# Dealing with merge conflicts



Git will **inform** you if a merge conflict happens when merging:

```
Auto-merging yikjin.md
CONFLICT (content): Merge conflict in yikjin.md
Automatic merge failed; fix conflicts and then commit the result.
```



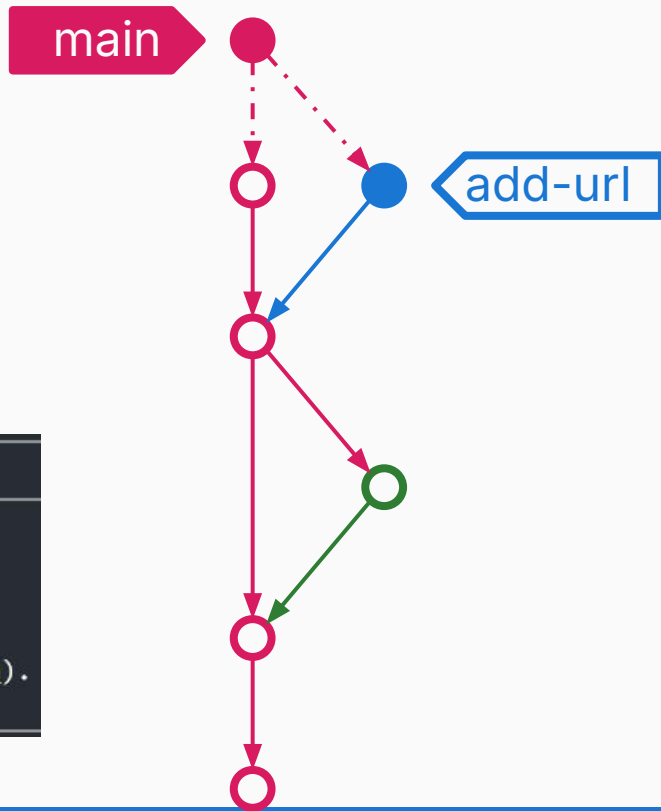
# Dealing with merge conflicts



Open the conflicting file(s)

- First part comes from **current/destination** branch
- Second part comes from **source** branch

|   | File: yikjin.md                                                |
|---|----------------------------------------------------------------|
| 1 | # Yik Jin                                                      |
| 2 |                                                                |
| 3 | <<<<<< HEAD                                                    |
| 4 | Hello! Nice to meet you!                                       |
| 5 | =====                                                          |
| 6 | Hello! This is my [GitHub profile](https://github.com/yikjin). |
| 7 | >>>>>> add-url                                                 |



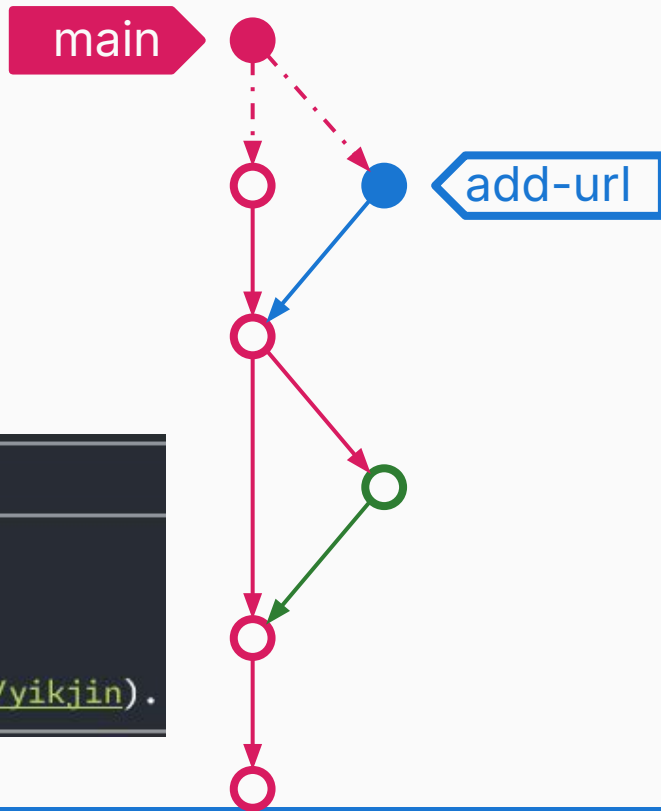
# Dealing with merge conflicts



**Update** the file(s) until they are how you want it to be

- This means **removing all the formatting marks** inserted by Git as well!

|   |                                                                                                   |
|---|---------------------------------------------------------------------------------------------------|
|   | File: yikjin.md                                                                                   |
| 1 | # Yik Jin                                                                                         |
| 2 |                                                                                                   |
| 3 | Hello! Nice to meet you!                                                                          |
| 4 |                                                                                                   |
| 5 | This is my [GitHub profile]( <a href="https://github.com/yikjin">https://github.com/yikjin</a> ). |

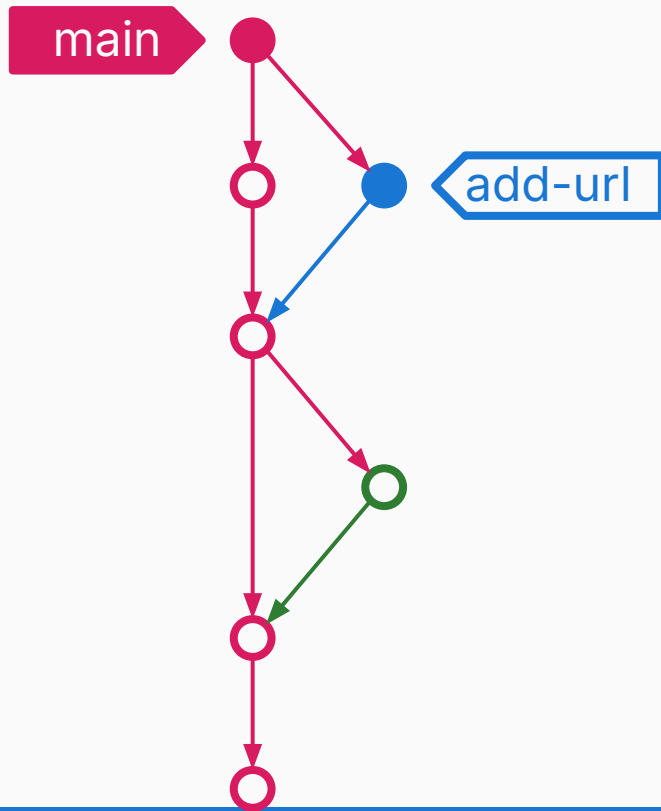


# Dealing with merge conflicts



Tell Git that you are done resolving the merge conflict, and to **continue merging**, with `git add {file}` and then `git merge --continue`

If you **don't want to merge**, use `git merge --abort` instead



# Activity 6: Merge conflicts



Time to instigate a merge conflict, and learn to resolve it:

1. **Create a branch** (*Branch A*) from "main", and **change a line** in a file
2. **Switch** to "main" branch
3. **Create another branch** (*Branch B*) from "main", and **change the same line** in the same file
4. **Switch** to "main" branch
5. **Merge** *Branch A* into "main", then *Branch B* into "main"
6. **Resolve** the resulting merge conflict, and **verify** results afterwards

# Advanced: Undoing in Git



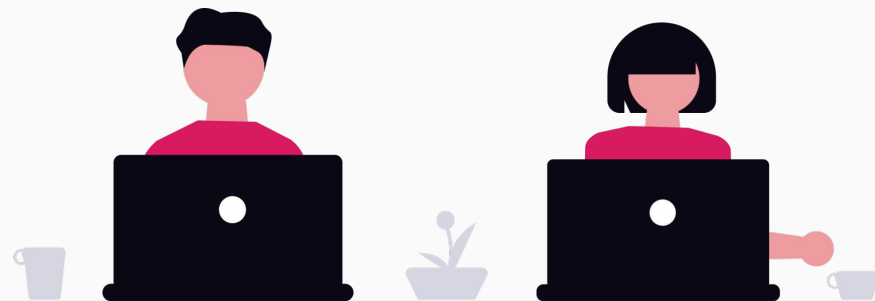
There is **no generic undo command** in Git, instead you just run Git commands that end up effectively undoing what you did

- To **unstage** a staged file, run `git restore -S {file}`
- To **undo a commit**, run `git reset --hard {commit}` or `git reset --hard HEAD~{n}` to go back N commits

***Not recommended*** to undo commits in Git, as it changes the history, and can screw things up if you are not careful

# Intro to GitHub

Get these slides at [hckr.cc/hs-git-intro](https://hckr.cc/hs-git-intro)



# What is GitHub?



GitHub is an **online service** that allows for **collaboration** with others using Git DVCS

- Online server that stores your Git repository, so others can access it and contribute code as well



# Create a GitHub online repository



Go to GitHub, and **create a repository** using the button  New

- Enter the **repository name**, and optionally the description
- Keep the repository visibility **public**
- Once done, click 

# Upload local repository to GitHub



Next, you need to tell your local Git repository that the online repository exists, done by setting a **Git remote**

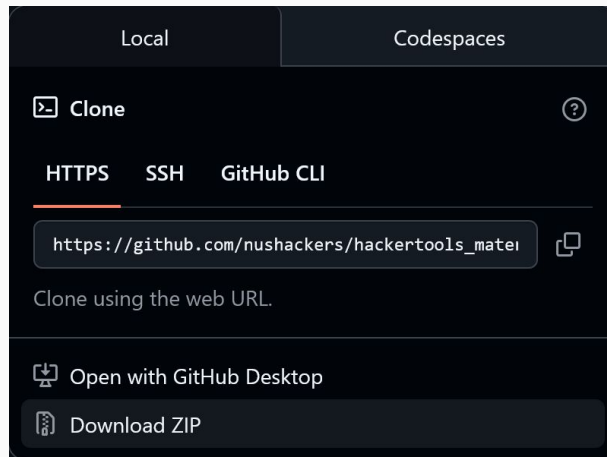
- Remotes are **remote code repositories** that Git can talk to

# Upload local repository to GitHub



To **upload** your local repository to GitHub:

1. Find out the **repository clone URL** by clicking on `<> Code` and copying the HTTPS URL
2. In your local repository, **run**  
`git remote add {remote-name} {url}`
  - By convention, `{remote-name}` is usually "**origin**" for your main online repository



# Upload local repository to GitHub



Ensure the remote is **successfully added** with

```
git remote -v
```

```
origin https://github.com/yikjin/intro-to-git.git (fetch)
origin https://github.com/yikjin/intro-to-git.git (push)
```

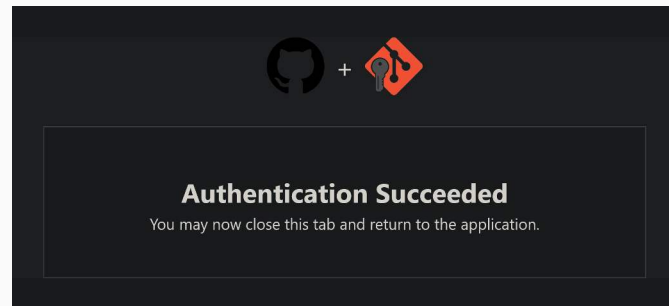
# Upload local repository to GitHub



Uploading code to a remote is known as  
**“pushing” code to the remote**

- Run `git push -u origin main` to push for the **first time**
- Enter your GitHub credentials and authorise Git Credential Manager to your GitHub account by clicking

Authorize git-ecosystem



# Upload local repository to GitHub



To **check if the push succeeded**, you should see something like this:

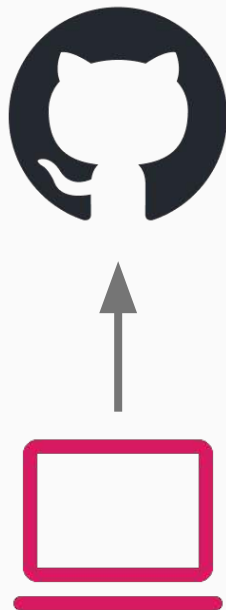
```
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (19/19), 1.91 KiB | 490.00 KiB/s, done.
Total 19 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/yikjin/intro-to-git.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.
```

# Subsequent sync with GitHub



After the first push, you can **push subsequent changes** to GitHub with one of:

- `git push` (push **current** branch to "origin")
- `git push --all` (push **all** branches to "origin")
- `git push {remote-name} {branch}`  
(push {branch} to {remote-name})

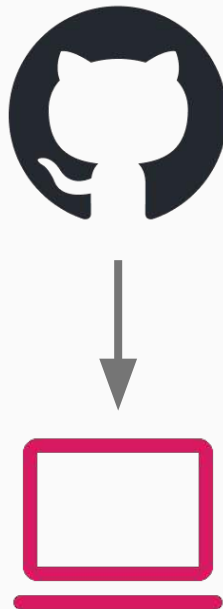


# Subsequent sync with GitHub



You can get ("pull") **subsequent changes** from GitHub with one of:

- `git pull` (pull **current** branch from "origin")
- `git pull --all` (pull **all** branches from all remotes)
- `git pull {remote-name} {branch}`  
(pull {branch} from {remote-name})



# Recommended practices for remotes



**Regularly pull** from your remote to get changes from others

**Do not override history** (your past commits) once they have been pushed to remotes

Work on your **own branches** to minimise conflicts when pulling or merging commits

# Add collaborators on GitHub



You can **add collaborators** to your repository by going to *your GitHub repository > Settings > Collaborators*, and **adding the username** of your other team members

- Without collaborator access, others **cannot** push to your GitHub repository, even if it is public!

# How real-life projects work



In the real world, you **can't** assign collaborator access to everyone

- It is **not safe** to add collaborators that you do not trust, since they can simply override commits

Real-life projects use **pull requests** for others to propose code changes that can only be merged by approved people

# Further reading



We can't cover everything in a session, so here's more **essential stuff** to read up on:

1. [GitHub pull requests](#)
2. [GitHub issue tracker](#)

**Useful things** that may interest you:

1. [Official Git book](#)
2. [Learn Git Branching](#)
3. [GitHub documentation](#)

# Need more help?



Come for Hackerspace **tomorrow, 4 – 6pm at COM1-0216!**

- **Chill social session** where you can hang out, make friends, and learn new stuff





# Thank you for coming!

See you again soon for our other events!

Get these slides at [hckr.cc/hs-git-intro](https://hckr.cc/hs-git-intro)

